



University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE – COAL Lab Project Report

PASSWORD-PROTECTED LOGIN SYSTEM

A Console-Based Multi-User Authentication System

Developed in 32-bit C++ with Inline x86 Assembly (MASM)

1. OVERVIEW

1.1 Project Title

Password-Protected Login System

(Console application displayed as "Password Protection Login System" at runtime)

1.2 Objective

The **primary** objective of this project is to design and implement a fully **functional**, console-based, multi-user authentication system using C++ integrated with **inline x86** assembly language. The system enforces strong **security** policies including account locking, administrative controls, password strength **validation**, session-based password expiry, and login streak tracking, all within a **32-bit** Windows environment.

1.3 System Flow

- Program launches, loads existing user data from disk
- Main menu presents three options: Admin Login, User Login, Exit
- **Admin path:** credential verification → Admin Panel with full user management
- **User path:** sub-menu with Login, Register, Forgot Password options
- **Successful user login:** User Panel with profile, settings, activity, and history
- **Logout or Exit:** system data is saved back to disk

1.4 Technology Stack

Component	Detail
Language	C++ with Inline __asm Blocks
Assembly Dialect	x86 MASM-compatible inline assembly
Platform	Windows, Visual Studio
Storage	Plain-text flat file (users.txt)
I/O	Console (conio.h, Windows Console API)
Libraries	iostream, string, ctime, windows.h, conio.h

2. DATA STRUCTURES & MEMORY LAYOUT

2.1 UserRecord Structure

Each **registered** user is represented by a UserRecord struct containing all necessary **authentication** and session metadata:

Field	Type	Size	Purpose
name	char[]	30 bytes	Unique username
pass	char[]	30 bytes	Plain-text password

Field	Type	Size	Purpose
secQuestion	char[]	60 bytes	Security question text
secAnswer	char[]	30 bytes	Answer for password recovery
logins	int	4 bytes	Total successful logins count
failed	int	4 bytes	Cumulative failed login attempts
locked	int	4 bytes	Temporary lock flag (0/1)
blocked	int	4 bytes	Admin-imposed permanent block flag
lockTime	time_t	4/8 bytes	Timestamp when lock was applied
loginExpiry	int	4 bytes	Login count toward password expiry (max 5)
lastLogin	char[]	30 bytes	Timestamp string of most recent login
history[5]	char[][]	150 bytes	Rolling last-5 login timestamp records
historyCount	int	4 bytes	Number of history entries stored
lastLoginDate	char[]	12 bytes	Date string for streak calculation
currentStreak	int	4 bytes	Current consecutive daily login streak
bestStreak	int	4 bytes	All-time best streak
passReminder	int	4 bytes	Reserved for future reminder flag

2.2 SystemData Structure

A single SystemData instance manages the entire application state:

Field	Type	Purpose
users[20]	UserRecord[]	Array of up to 20 user records
TotalUsers	int	Current number of registered users
TotalLogins	int	System-wide cumulative login counter
LastLoggedUser	char[30]	Username of the last successful login
AdminUser	char[30]	Administrator username
AdminPass	char[30]	Administrator password
AdminLocked	int	Admin temporary lockout flag
AdminLockTime	time_t	Timestamp when admin lock was applied

2.3 Memory Allocation Strategy

- All user **records** reside in the static SystemData struct on the **stack/data** segment
- Persistent **storage** is handled via a plain-text file (users.txt) read at startup and written on every modification
- Maximum capacity is **20** simultaneous users

- Inline assembly routines **operate** directly on the struct fields through **pointer** manipulation

3. INLINE ASSEMBLY FUNCTIONS

The program incorporates **17 inline x86** assembly routines, each implementing a specific low-level operation. All functions use the `__asm` block syntax compatible with MSVC's 32-bit compiler.

ASM #	Function Name	Description	Key Registers Used
1	compareStr()	Character-by-character string comparison; returns 1 if equal, 0 otherwise	ESI, EDI, AL, BL
2	addOne()	Increments an integer via pointer using INC instruction	EAX
3	takeOne()	Decrements an integer via pointer using DEC instruction	EAX
4	isUpper()	Checks if a character is an uppercase letter (ASCII 65-90)	EAX
5	isLower()	Checks if a character is a lowercase letter (ASCII 97-122)	EAX
6	isDigit()	Checks if a character is a decimal digit (ASCII 48-57)	EAX
7	isSpecial()	Checks if a character falls in printable special-character ranges	EAX
8	asmStrLen()	Computes string length using REPNE SCASB scan instruction	EDI, ECX, EAX
9	asmStrCopy()	Copies a string byte-by-byte using LODSB/STOSB	ESI, EDI, AL
10	asmCheckLock()	Compares elapsed time against lock duration; returns lock status	EAX, EBX, ECX
11	asmBubbleSwap()	Swaps two UserRecord structs using REP MOVSB block copy	ECX, ESI, EDI
12	asmBeepSuccess()	Emits a high-frequency double beep via Windows Beep() call	Stack
13	asmBeepError()	Emits a low-frequency double beep for error feedback	Stack
14	asmBeepWarning()	Emits a medium single beep for warning feedback	Stack
15	asmCountChar()	Counts occurrences of a character in a string using loop/compare	ESI, EBX, ECX, EAX
16	asmStrCat()	Concatenates one string onto another using LODSB/STOSB	EDI, ESI, AL
17	asmToUpperChar()	Converts a single lowercase character to uppercase (SUB eax, 32)	EAX

3.1 Notable Assembly Techniques

String Comparison

Uses ESI and EDI pointers to walk both strings simultaneously, comparing bytes with **CMP AL, BL**. The loop terminates either at a mismatch or at the null terminator.

String Length with SCASB

Loads **ECX** with 0FFFFh (max search count), sets **AL = 0** (null terminator), then executes **REPNE SCASB** to scan until null. The length is derived as 0FFFFh **minus** the remaining **ECX**, minus one.

Struct Swap with REP MOVSB

The `asmBubbleSwap` function performs a three-way copy (A to temp, B to A, temp to B) using **REP MOVSB** with **ECX** set to `sizeof(UserRecord)`, enabling efficient block memory moves **essential** for user removal and reordering.

Lock/Block Flag Setting via Pointer

Throughout the program, account lock and block flags are set directly via pointer dereferences inside inline `__asm` blocks: **MOV EAX, ptr; MOV DWORD PTR[EAX], 1**, illustrating direct memory write without C++ intermediaries.

Progress Bar Division

An inline assembly block computes the filled bar width using **IMUL** followed by **IDIV ECX** (divide by max), storing the quotient in `filled`, demonstrating integer arithmetic in assembly.

4. FUNCTIONAL MODULES — DETAILED ANALYSIS

4.1 System Initialization

The `initSystem()` function zeroes all `UserRecord` fields, sets the default admin credentials (admin / Admin@1234), and calls `loadUsers()` to restore persisted data from `users.txt`. This function is called once at program startup.

4.2 File Persistence (saveUsers / loadUsers)

- **saveUsers():** Opens `users.txt` in write mode and serialises the `SystemData` header (`totalUsers`, `totalLogins`, `lastLoggedUser`) followed by each `UserRecord` as newline-delimited fields, ending each record with a `---` separator.
- **loadUsers():** Reads and deserialises the same format using `fgets()` and `atoi()/atoll()`, rebuilding the in-memory `SystemData` on startup.
- File is rewritten on every write operation (registration, login, block, remove, password change) ensuring data **consistency**.

4.3 Admin Authentication

- **Default credentials:** username admin, password Admin@1234
- **Three password** attempts are allowed before a 10-second lockout
- **Username** must be verified before the password prompt is shown
- **Optional** password visibility toggle (y/n prompt before entry)

4.4 Admin Panel — Functions

Option	Function	Description
1	View Total Users	Displays count with a visual progress bar
2	Total System Logins	Shows cumulative login counter for all users
3	View All Users	Tabular list: username, logins, failed, status, blocked flag
4	Search User	adminSearchUser() — find and display full user record
5	Remove User	adminRemoveUser() — confirm then shift array and decrement count
6	Login Audit Report	adminAuditReport() — tabular view: logins, failures, activity, streak, blocked
7	View Login History	adminViewLoginHistory() — last 5 logins for a named user
8	Block User	adminBlockUser() — set blocked = 1 via inline ASM pointer write
9	Unblock User	adminUnblockUser() — set blocked = 0 and clear locked flag
10	Logout	Return to main menu

4.5 User Registration

- Validates unique, non-empty username
- Presents four security question choices; answer is stored in secAnswer
- Option to auto-generate a strong 10-character password (2 upper, 3 lower, 3 digits, 2 special, shuffled)
- Manual password entry enforced through checkPasswordStrength(): Weak (score ≤ 1 or length < 6) → rejected; Medium (score = 2 or length < 8) → accepted with warning; Strong (score ≥ 3 and length ≥ 8) → fully accepted
- Password confirmation required; mismatch loops back to re-entry
- All fields initialised to zero via inline ASM pointer writes; record saved immediately

4.6 User Login

- Username look-up via findUser() using compareStr()
- Blocked check:** account blocked by admin → login refused with guidance
- Temporary lock check:** if locked flag is set, asmCheckLock(30s) is evaluated; if expired, lock is cleared via inline ASM
- Three password attempts; on exhaustion: locked = 1, lockTime = time(NULL), 30-second lockout enforced
- On success:** logins++, totalLogins++, loginExpiry++, login history updated, streak updated
- Password expiry:** at loginExpiry = 4 a reminder is shown; at 5 the user is forced to set a new password before proceeding

4.7 Password Recovery (Forgot Password)

- User enters username; blocked accounts are denied
- Security question is presented; answer compared with compareStr()
- On correct answer: new password entered, confirmed, strength-checked, saved; locked and loginExpiry reset to 0 via inline ASM

4.8 User Panel

Option	Function	Description
1	View Profile	Full user stats: activity level, logins, failures, last login, streak, status, expiry counter
2	Settings	Change Username / Change Password / Delete Account
3	Account Activity Level	Visual activity scale with progress bar (Beginner / Regular / Power User)
4	Login Streak	Daily consecutive login streak display with motivational feedback
5	View Login History	Last 5 login timestamps from rolling history buffer
6	Logout	Returns to user sub-menu

4.9 Settings Sub-Menu

Change Username

Requires current password **verification** (compareStr). New username must be **non-empty** and unique (findUser check). **Updated** in-place using asmStrCopy.

Change Password

Requires current password. New password must pass **strength check** and confirmation match. loginExpiry reset to 0 via inline ASM after change.

Delete Account

Requires correct password, then the user must type **DELETE** literally. Confirmed deletion shifts subsequent records using asmBubbleSwap and decrements totalUsers.

5. ALGORITHMS & LOGIC

5.1 Password Strength Algorithm

The checkPasswordStrength(char* pass) function evaluates four Boolean criteria using assembly-based character classifiers:

- **hasUpper** — at least one character satisfies isUpper() (ASCII 65–90)
- **hasLower** — at least one character satisfies isLower() (ASCII 97–122)
- **hasDigit** — at least one character satisfies isDigit() (ASCII 48–57)
- **hasSpecial** — at least one character satisfies isSpecial() (printable non-alphanumeric ranges)

5.2 Password Generation Algorithm

generatePassword(char* out) constructs a **10-character** password from four-character pools:

- 2 uppercase letters from 'A'–'Z'
- 3 lowercase letters from 'a'–'z'
- 3 decimal digits from '0'–'9'
- 2 special characters from '!'@#\$\$%^&*

5.3 Login Streak Tracking

updateStreak() converts the current date and last login date strings (**DD/MM/YYYY**) to integer day-counts using a simplified formula: **Y×365 + M×30 + D**. The difference determines:

- diff = 0:** Same-day login — no change to streak
- diff = 1:** Consecutive day — currentStreak++ and bestStreak updated if exceeded
- diff > 1:** Streak broken — currentStreak resets to 1

5.4 Activity Level Classification

getActivityLevel(int logins) maps login count to a user tier:

Tier	Condition	Label
Beginner	logins < 5	New / infrequent user
Regular	5 ≤ logins < 20	Consistent user
Power User	logins ≥ 20	Highly active user

5.5 Temporary Lockout Mechanism

asmCheckLock(time_t lockTime, int seconds) computes elapsed = **now - lockTime** using x86 SUB and compares to the threshold with CMP/JGE. Returns 1 if still locked, 0 if expired. User lockout threshold: 30 seconds. Admin lockout threshold: 10 seconds.

5.6 Rolling Login History

addLoginHistory() maintains a circular buffer of **5** entries. When historyCount < 5 the timestamp is appended; when full, entries 0–3 are shifted down using asmStrCopy and the new entry occupies index 4, preserving the most recent five logins.

6. SECURITY FEATURES

Feature	Implementation Detail
Account Lockout (User)	3 failed password attempts → 30-second temporary lock via time_t comparison in ASM
Account Lockout (Admin)	3 failed password attempts → 10-second temporary lock
Admin Block (Permanent)	Admin sets blocked = 1 via direct pointer write; user cannot login until unblocked
Password Strength Enforcement	Minimum criteria enforced at registration, password change, and recovery
Password Expiry	Passwords expire after 5 successful logins; change enforced before session continues
Security Question Recovery	Users answer a selected security question to reset their forgotten password
Password Masking	Default input hides characters with '*'; optional visible mode available

Feature	Implementation Detail
Username Uniqueness	findUser() scan ensures no duplicate usernames at registration or rename
Deletion Confirmation	Account deletion requires typing 'DELETE' explicitly in addition to password verification
Audit Logging	Admin can view per-user login counts, failure counts, activity level, streak, and block status

7. SYSTEM API & CONSOLE FUNCTIONS

Function / API	Usage in Program
GetStdHandle / SetConsoleTextAttribute	setColor() function for colour-coded console output
GetLocalTime (SYSTEMTIME)	getCurrentTime() and getCurrentDate() for timestamps
Beep() (Windows API)	Called from inline ASM for audio feedback (success/error/warning)
_getch() (conio.h)	Non-echoing single-character input for menus and password toggle
time() / difftime() (ctime)	Lock duration measurement in lockout system
fopen_s / fgets / fprintf / fclose	File I/O for loading and saving user records
SetConsoleTitleA	Sets the console window title at startup
Sleep() (windows.h)	Brief pauses for user feedback (600–1000 ms)
system("cls")	Screen clearing between menu transitions

8. EXECUTION FLOW DIAGRAM

The following represents the complete top-level execution flow of the Password-Protected Login System:

Step	Action	Outcome
1	main() — SetConsoleTitle, initSystem()	Load users.txt, initialise SystemData
2	Main Menu Display	Three choices: Admin Login / User Login / Exit
3	Admin Login Selected	Verify admin credentials with lockout
4	Admin Panel	10-option management panel (view, search, remove, block, unblock, audit, history)
5	User Login Selected	Sub-menu: Login / Register / Forgot Password
6	Register	Validate name uniqueness, select security Q, set password, save
7	Login	Verify credentials, check locks, update stats, enter User Panel

Step	Action	Outcome
8	Forgot Password	Verify username, answer security Q, reset password
9	User Panel (post-login)	Profile / Settings / Activity / Streak / History / Logout
10	Exit	Beep warning, farewell message, program terminates

9. LIMITATIONS & POTENTIAL IMPROVEMENTS

9.1 Current Limitations

- Passwords stored in plain text in users.txt; no hashing or **encryption**
- Maximum user capacity hard-coded at **20**
- Windows-only due to **dependence** on windows.h, conio.h, and Visual Studio MSVC __asm
- File format **is line-delimited** text, susceptible to manual editing
- No **multi-session** or concurrent access support
- Security question answers stored in plain text with exact-match comparison

9.2 Suggested Enhancements

- Implement SHA-256 or Ecrypt password hashing for secure storage
- Replace flat-file storage with a binary or structured database format
- Add ANSI escape code support for cross-platform compatibility
- Implement case-insensitive security answer comparison
- Add an "Admin Change Password" option within the Admin Panel
- Introduce CAPTCHA-style delay scaling with failed attempts
- Port inline ASM blocks to NASM for 64-bit Linux compatibility
- Implement email-based OTP as an additional password recovery channel

10. TECHNICAL SPECIFICATIONS

Parameter	Value
Platform	Windows (32-bit)
Compiler	Microsoft Visual C++, 32-bit mode
Assembly	x86 Inline __asm
Build Mode	Release / Debug — 32-bit (x86)
Storage File	users.txt
Max Users	20
Password History Buffer	5 entries
User Lockout Duration	30 seconds

Parameter	Value
Admin Lockout Duration	10 seconds
Password Expiry Threshold	5 successful logins
Inline ASM Functions	17
Password Min Length (enforced)	6 characters
Generated Password Length	10 characters

11. EDUCATIONAL VALUE

This project delivers hands-on learning across multiple disciplines of systems programming and computer architecture:

- **Low-level Memory Manipulation:** Direct struct field access through pointer arithmetic in inline ASM
- **x86 Register Usage:** Practical use of EAX, EBX, ECX, EDX, ESI, EDI, and AL/BL/CL in real-world scenarios
- **String Processing in Assembly:** LODSB, STOSB, REPNE SCASB, and REP MOVSB for efficient string and memory operations
- **Arithmetic in ASM:** INC, DEC, IMUL, IDIV used for counter management and progress bar calculation
- **Jump and Branching Logic:** Conditional jumps (JL, JG, JE, JNE, JGE) implementing range checks and comparisons
- **C++/ASM Integration:** Seamless blending of high-level C++ control flow with low-level assembly subroutines
- **Security Design:** Practical implementation of lockout policies, password expiry, and admin control hierarchies
- **File I/O:** Serialisation and deserialisation of complex structured data to and from flat-file storage
- **User Interface:** Colour-coded, interactive console UI using the Windows Console API

12. CONCLUSION

The Password-Protected Login System is a comprehensive **COAL** project that successfully integrates **17 inline x86** assembly functions within a **32-bit C++** application to deliver a fully operational, multi-user authentication platform.

The project demonstrates mastery of **core assembly language concepts** including register manipulation, string processing with SCASB/MOVSB instructions, arithmetic operations, conditional branching, and direct memory access — all applied within real **security-relevant** contexts such as account lockout, password validation, and data persistence.

Key learning outcomes achieved:

- Direct application of x86 registers and instruction set in a practical system

- Integration of assembly routines with C++ data structures and control flow
- Implementation of security policies (lockout, expiry, admin control) at the system level
- Persistent data management via structured file I/O
- Console-based UI design with colour-coded feedback and audio cues

The system functions reliably within its defined scope and provides a strong foundation for further enhancement through encryption, cross-platform support, and expanded administrative features.